

Mathematical Foundations of DCA-Trie: A Complete Guide to Notation, Definitions, and Proofs

Bernard

Department of Computer Science, University of Mines and Technology

July 27, 2026

Abstract

This document provides a self-contained mathematical guide to the DCA-Trie project. Every symbol, formula, definition, and theorem used across the codebase is explained from first principles. The goal is to enable anyone to read the paper, the code, or the article and understand exactly what each piece of math means, why it is correct, and how to derive it.

1 Graph Theory

1.1 Knowledge graph as a directed edge-labeled multigraph

Definition 1.1 (Knowledge Graph). A *knowledge graph* is a tuple $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ where:

- $\mathcal{E}$ is a finite set of **entities** (nodes),
- $\mathcal{R}$ is a finite set of **relations** (edge labels),
- $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ is a set of **triples** (directed labeled edges).

A triple $(h, r, t) \in \mathcal{T}$ asserts that entity h (the *head*) relates to entity t (the *tail*) via relation r . Freebase bollacker2008freebase has $|\mathcal{E}| \approx 40\text{M}$ entities, $|\mathcal{R}| \approx 350\text{M}$ relations.

Why a multigraph? Two entities can be connected by multiple relations: (h, r_1, t) and (h, r_2, t) are distinct triples. The “multi” in multigraph means multiple edges between the same node pair are allowed, each with a different label.

1.2 Subgraph extraction

Definition 1.2 (L -hop subgraph). Given a topic entity $E_q \in \mathcal{E}$ and hop limit $L \in \mathbb{N}$, the **L -hop subgraph** is:

$$\mathcal{G}_{E_q, L} = (\mathcal{E}_L, \mathcal{R}_L, \mathcal{T}_L)$$

where $\mathcal{E}_L = \{e \in \mathcal{E} : d(E_q, e) \leq L\}$, d is the shortest-path distance in the unweighted graph, and $\mathcal{T}_L = \mathcal{T} \cap (\mathcal{E}_L \times \mathcal{R} \times \mathcal{E}_L)$.

In code, this is computed by `nx.all_simple_edge_paths(graph, source, target, cutoff=L)` or BFS from the topic entity.

1.3 Path enumeration

Definition 1.3 (Reasoning path). A *reasoning path* of length L is a sequence:

$$p = (e_0, r_1, e_1, r_2, e_2, \dots, r_L, e_L)$$

where $(e_{i-1}, r_i, e_i) \in \mathcal{T}$ for all $i = 1, \dots, L$.

The path set $\mathcal{P}_{\text{GCR}}$ is the set of all reasoning paths of length exactly L reachable from E_q within the subgraph.

Path explosion. The number of paths grows as $O(E^L)$ where E is the average branching factor. For Freebase subgraphs with $E \approx 20$ and $L = 3$, this gives $20^3 = 8,000$ paths. For $L = 4$, it is 160,000. This is the fundamental scalability challenge.

1.4 Graph operations in code

```
1 import networkx as nx
2
3 # Build directed graph from triples
4 G = nx.DiGraph()
5 for h, r, t in triples:
6     G.add_edge(h, t, relation=r)
7
8 # All simple paths of length exactly L
9 paths = []
10 for target in G.nodes():
11     for path in nx.all_simple_edge_paths(G, source, target, cutoff=L):
12         if len(path) == L:
13             paths.append(path)
14
15 # Neighbors of an entity (for v2 dynamic expansion)
16 neighbors = list(G.neighbors(entity))
```

Listing 1: Graph operations from `src/utils/graph_utils.py`

2 Type Systems and Ontology

2.1 Entity types

Definition 2.1 (Entity type). An *entity type* is a categorical label assigned to an entity. For entity $e \in \mathcal{E}$, its type set is $\text{types}(e) \subseteq \mathcal{C}$, where $\mathcal{C}$ is a universe of type constants.

In Freebase, types come from `common.topic.notable_types`. Examples:

Type constant	Example entities
Person	Albert Einstein, Barack Obama
Location	Germany, Washington D.C.
Film	Inception, Titanic
Organization	NASA, Harvard University

In code, types are represented as immutable sets:

```

1 from typing import FrozenSet
2
3 _PERSON_TYPES: FrozenSet[str] = frozenset({
4     "Person", "Deceased Person", "Politician",
5     "Musical Artist", "Author", ...
6 })
7
8 def entity_types(entity_name: str) -> FrozenSet[str]:
9     """Look up entity types from KG metadata."""
10    return self._entity_types.get(entity_name, frozenset())

```

Listing 2: Type representation from `type_oracle.py`

2.2 Relation schema

Definition 2.2 (Relation schema). *For relation $r \in \mathcal{R}$, the **schema** specifies:*

- $\text{domain}(r) \subseteq \mathcal{C}$: *expected types of the head entity,*
- $\text{range}(r) \subseteq \mathcal{C}$: *expected types of the tail entity.*

Example: $\text{range}(\text{people.person.nationality}) = \{\text{Location}, \text{Country}, \dots\}$. This means the tail entity of a “nationality” relation should be a location.

Why this matters. If a candidate path includes `Einstein -> nationality -> Physics`, the range gate catches it: `Physics` is a `CreativeWork`, not a `Location`. The path is pruned.

2.3 Answer type inference

Definition 2.3 (Answer type set). *For question q , the **answer type set** $\mathcal{T}(q) \subseteq \mathcal{C}$ is the set of expected entity types for the correct answer. Inferred via 19 regex patterns:*

$$\mathcal{T}(q) = \bigcup_{i=1}^{19} \text{match}_i(q)$$

where $\text{match}_i(q)$ returns a frozenset of types if pattern i matches q , and $\emptyset$ otherwise.

Examples:

Pattern	Question	$\mathcal{T}(q)$
“who”	Who directed Inception?	{Person}
“where”	Where is Paris?	{Location}
“country”	What country is Paris in?	{Country, Location}

3 Constrained Decoding

3.1 The decoding problem

Definition 3.1 (Autoregressive generation). *An LLM generates text token-by-token. At step t , given tokens $\sigma_{1:t-1} = (t_1, \dots, t_{t-1})$, the model produces a probability distribution over the vocabulary V :*

$$P(t \mid \sigma_{1:t-1}) = \text{softmax}(W \cdot h_{t-1})$$

where h_{t-1} is the hidden state and $W \in \mathbb{R}^{|V| \times d}$ is the output projection.

Definition 3.2 (Constrained decoding). ***Constrained decoding** restricts the set of admissible tokens at each step. Instead of sampling from the full vocabulary V , the model is restricted to a subset $W_t \subseteq V$:*

$$P_{\text{constrained}}(t \mid \sigma_{1:t-1}) = \begin{cases} P(t \mid \sigma_{1:t-1})/Z & t \in W_t \\ 0 & t \notin W_t \end{cases}$$

where $Z = \sum_{t' \in W_t} P(t' \mid \sigma_{1:t-1})$ is the normalization constant.

In code, this is implemented via HuggingFace’s `prefix_allowed_tokens_fn` callback:

```

1 class GraphConstrainedDecoding:
2     def allowed_tokens_fn(self, batch_id, sent):
3         # Check if we're inside a <PATH>...</PATH> block
4         constrained_flag, L_input = self.check_constrained_flag(sent)
5
6         if constrained_flag:
7             # Query trie for valid next tokens
8             allow_tokens = self.trie.get(sent[L_input:].tolist())
9             if len(allow_tokens) == 0:
10                 return self.all_tokens # fallback
11             return allow_tokens
12         else:
13             return self.all_tokens # unconstrained

```

Listing 3: Constrained decoding callback from `graph_constrained_decoding.py`

3.2 The trie

Definition 3.3 (Trie). *A **trie** (prefix tree) is a tree data structure where each node represents a token ID. The root is the empty sequence. A path from root to a leaf represents a complete token sequence. The trie supports:*

- ***insert(sequence)**: add a token sequence, $O(n)$ where n is sequence length.*
- ***get(prefix)**: return all valid next tokens given a prefix, $O(n)$ where n is prefix length.*

Why a trie? The trie encodes the set of valid paths as a prefix tree. During decoding, `get(prefix)` returns exactly the set of tokens that would keep the generated sequence on a valid path. This is $O(1)$ per token in practice (constant branching factor).

MarisaTrie. We use `marisa-trie`, a C-backed implementation using minimal perfect hashing. Each token ID i is mapped to a Unicode code point:

$$\text{char}(i) = \text{chr}(0xE000 + i)$$

This produces a string that the C backend can index efficiently. The mapping is injective (different token IDs map to different characters).

3.3 The prefix matching problem

Definition 3.4 (Prefix match). *Given a generated sequence $\sigma_{1:t}$ and a position L_{input} (the start of the generated portion), the **prefix** is $\sigma_{L_{\text{input}}:t}$. The trie returns:*

$$W_t = \text{trie.get}(\sigma_{L_{\text{input}}:t})$$

This is the critical constraint: the trie matches from L_{input} onward. If the prompt and trie disagree on where L_{input} is, the constraint fails silently. This is why prompt-trie alignment matters.

4 Constraint Oracles

4.1 Formal framework

Definition 4.1 (Constraint oracle). A **constraint oracle** is a function $O : \mathcal{P} \rightarrow \{0, 1\}$ that assigns each candidate path a binary admission decision. $O(p) = 1$ means the path is admitted; $O(p) = 0$ means it is excluded.

Definition 4.2 (Oracle tightness). Given a candidate path set $\mathcal{P}$ and oracle O , the **oracle tightness** is:

$$\text{tightness}(O) = 1 - \frac{|\{p \in \mathcal{P} : O(p) = 1\}|}{|\mathcal{P}|}$$

subject to the recall constraint $O(p_q^*) = 1$ for all gold-truth paths p_q^* .

Tightness measures the fraction of paths excluded. A tightness of 0 admits everything; a tightness of 1 admits nothing.

Derivation. The fraction of admitted paths is $|\{p : O(p) = 1\}|/|\mathcal{P}|$. Tightness is 1 minus this fraction. It is a number in $[0, 1]$.

4.2 Oracle hierarchy

Proposition 4.3 (Oracle hierarchy). The following containment holds:

$$O_{GCR} \subseteq O_{ont} \subseteq O_{question} \subseteq O_{useful}$$

where:

- O_{GCR} : structural oracle (admits all KG-valid paths),
- O_{ont} : ontology oracle (adds type filtering),
- $O_{question}$: question-specific oracle (adds entity linking),
- O_{useful} : the ideal oracle that admits only useful paths.

Each level adds a stricter admission criterion. We instantiate O_{ont} .

5 The Two Gates

5.1 Gate 1: Answer type gate

Definition 5.1 (Type gate). For entity e_h at hop h with max hops L and answer types $\mathcal{T}(q)$:

$$\text{type_gate}(e_h, q, h, L) = \begin{cases} \top & h < L \\ \top & \mathcal{T}(q) = \emptyset \\ \top & \text{types}(e_h) = \emptyset \\ \mathbf{1}[\text{types}(e_h) \cap \mathcal{T}(q) \neq \emptyset] & \text{otherwise} \end{cases}$$

Why four cases?

1. **Intermediate hop** ($h < L$): Not the terminal entity, so type compatibility does not apply. Pass.
2. **No answer types** ($\mathcal{T}(q) = \emptyset$): No regex matched. We cannot prune. Pass.
3. **No entity types** ($\text{types}(e_h) = \emptyset$): Metadata is missing. Conservative: pass.
4. **General case**: Check set intersection. If the entity's types overlap with the expected answer types, admit.

Derivation. The indicator function $\mathbf{1}[A]$ returns 1 if A is true, 0 otherwise. The condition $\text{types}(e_h) \cap \mathcal{T}(q) \neq \emptyset$ is a set intersection test. In Python:

```

1 def type_gate(self, entity_name, answer_types, hop, max_hop):
2     if hop < max_hop:                # Case 1
3         return True
4     if not answer_types:              # Case 2
5         return True
6     etypes = self._entity_types.get(entity_name, frozenset())
7     if not etypes:                   # Case 3
8         return True
9     return bool(etypes & answer_types) # Case 4: set intersection

```

Conservative default. When metadata is absent (cases 2 and 3), the gate passes. This bounds the false negative rate: we never prune a path because of missing data, only because of incompatible data.

5.2 Gate 2: Property range gate

Definition 5.2 (Range gate). *For relation r and tail entity e' :*

$$\text{range_gate}(r, e') = \begin{cases} \top & r \notin \mathcal{R}_{\text{schema}} \\ \top & \text{range}(r) = \emptyset \\ \top & \text{types}(e') = \emptyset \\ \mathbf{1}[\text{types}(e') \cap \text{range}(r) \neq \emptyset] & \text{otherwise} \end{cases}$$

The logic mirrors the type gate. The four cases handle: unknown relation (pass), no range constraint (pass), no entity types (pass), and the general intersection test.

Why this works. Freebase declares $\text{range}(\text{people.person.nationality}) = \{\text{Location}, \dots\}$. If a candidate tail entity has type $\{\text{Person}\}$, the intersection is empty, and the gate rejects it. This catches structurally valid but semantically wrong paths.

5.3 Composition of gates

Proposition 5.3 (Gate composition). *The final path set is:*

$$\mathcal{P}_{\text{final}} = O_{\text{type}} \circ O_{\text{range}}(\mathcal{P}_{\text{topo}})$$

where $\circ$ denotes function composition: first apply range gate, then type gate.

Derivation. A path $p = (e_0, r_1, e_1, \dots, r_L, e_L)$ is admitted iff:

1. Every hop passes the range gate: $\forall i \in \{1, \dots, L\} : \text{range_gate}(r_i, e_i) = \top$.
2. The terminal entity passes the type gate: $\text{type_gate}(e_L, q, L, L) = \top$.

The range gate is applied at every hop; the type gate only at the terminal hop. The order does not matter because the gates are independent (they check different conditions on different entities).

6 Algorithmic Complexity

6.1 Gate operations

Proposition 6.1 (Gate complexity). *Each gate operation is $O(1)$.*

Proof: The gate performs:

1. A dictionary lookup to get entity types: $O(1)$ average case (hash table).
2. A dictionary lookup to get relation range: $O(1)$.
3. A set intersection test: $O(\min(|A|, |B|))$. Since type sets have bounded size (typically $|\text{types}(e)| \leq 5$), this is $O(1)$.

6.2 V1: Static filtering

Proposition 6.2 (V1 complexity). *V1 construction is $O(|P| \cdot L)$ where $|P|$ is the number of enumerated paths and L is the hop depth.*

Proof: For each path, we apply L range gate checks (one per hop) and 1 type gate check (terminal hop only). Each check is $O(1)$. Total: $O(|P| \cdot L)$. The path count $|P|$ grows as $O(E^L)$, so V1 is exponential in L .

6.3 V2: Dynamic expansion

Proposition 6.3 (V2 complexity). *V2 construction is $O(L \cdot B \cdot d_{\text{avg}})$ where L is hop depth, B is beam width, and d_{avg} is the average out-degree.*

Proof: At each of L hops, for each of B beams, we enumerate d_{avg} neighbors and apply $O(1)$ gate checks. Total: $O(L \cdot B \cdot d_{\text{avg}})$. With $d_{\text{avg}} \approx 20$ and $L = 3$, this is $O(60B)$, three orders of magnitude less than V1's exponential path count.

6.4 Comparison

Method	Complexity	Practical cost
V1 (static)	$O(E^L \cdot L)$	4,102,833 paths
V2 (dynamic)	$O(L \cdot B \cdot d_{\text{avg}})$	$\sim 60B$ operations

For $L = 3$, $E = 20$, $B = 1$: V1 processes 8,000 paths; V2 processes 60 operations. The ratio is $\sim 133\times$.

7 Cosine Similarity and Embeddings

7.1 Sentence embeddings

Definition 7.1 (Sentence embedding). A **sentence embedding** is a vector $E(x) \in \mathbb{R}^d$ produced by a sentence transformer (e.g., *all-MiniLM-L6-v2*, $d = 384$). Similar sentences have nearby vectors.

7.2 Cosine similarity

Definition 7.2 (Cosine similarity). For vectors $u, v \in \mathbb{R}^d$:

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \cdot \|v\|} = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \cdot \sqrt{\sum_{i=1}^d v_i^2}}$$

Range: $[-1, 1]$. Value 1 means identical direction; 0 means orthogonal; -1 means opposite.

Why cosine, not Euclidean distance? Cosine similarity is invariant to vector magnitude. Two paths with the same semantic content but different lengths produce different-magnitude embeddings but similar directions. Cosine captures this; Euclidean distance does not.

7.3 Approach 1: Cosine oracle

Definition 7.3 (Cosine oracle). For path p and question q :

$$O_{\text{cosine}}(p) = \mathbf{1}[\cos(E(p), E(q)) \geq \tau]$$

where $E(p)$ is the path embedding and τ is the threshold.

In code, the path is serialized as a string "h r t" before encoding.

7.4 Approach 2: Decomposed product score

Definition 7.4 (Decomposed score).

$$\text{score}(r, e', q) = \rho_r(r, q) \cdot \rho_e(e', q) \cdot \rho_{\text{trail}}(r, e', q)$$

where:

- $\rho_r(r, q) = \cos(E(r), E(q_{\text{rel}}))$: relational relevance (entity-masked question),
- $\rho_e(e', q) = \mathbf{1}[\text{type}(e') \in \mathcal{T}(q, h)]$: hard type gate,
- $\rho_{\text{trail}}(r, e', q) = \cos(E(r \oplus e'), E(q))$: trajectory relevance.

The product is a weighted combination of three orthogonal signals: what the relation means, whether the entity type fits, and how the whole triple relates to the question.

Entity masking. The relational relevance ρ_r uses a masked question q_{rel} where entity names are replaced with [MASK]. This forces the encoder to focus on the relation semantics, not the entity mentions.

8 Metrics

8.1 Hits@1

Definition 8.1 (Hits@1). *A prediction p is correct iff the ground-truth answer entity appears as a substring in the predicted path:*

$$\text{hit}(p, a) = \mathbf{1}[a \in \text{normalize}(p)]$$

Hits@1 is the fraction of questions where at least one ground-truth answer is a substring of the prediction:

$$\text{Hits@1} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \max_{a \in \text{answers}(q)} \text{hit}(p_q, a)$$

Normalization: lowercase, strip punctuation, remove articles, collapse whitespace.

8.2 F1 score

Definition 8.2 (F1). *For a set of predictions $\mathcal{P}$ and ground-truth answers $\mathcal{A}$:*

$$\text{precision} = \frac{|\mathcal{P} \cap \mathcal{A}|}{|\mathcal{P}|}, \quad \text{recall} = \frac{|\mathcal{P} \cap \mathcal{A}|}{|\mathcal{A}|}, \quad F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

8.3 Recall@K

Definition 8.3 (Recall@K). *Given a ranked list of paths and gold-truth indices $\mathcal{G}$:*

$$\text{Recall@K} = \frac{|\{g \in \mathcal{G} : g \in \text{top-}K\}|}{|\mathcal{G}|}$$

This measures whether the gold path appears in the top- K candidates. Used in the learned pruning experiments.

8.4 False negative rate (FNR)

Definition 8.4 (FNR).

$$\text{FNR} = 1 - \text{gold-path retention} = 1 - \frac{|\{p^* : O(p^*) = 1\}|}{|\{p^*\}|}$$

This is the fraction of gold-truth paths incorrectly pruned by the oracle. Lower is better. Our oracle achieves $\text{FNR} < 5\%$.

8.5 Selectivity-Irrelevance Ratio (SIR)

Definition 8.5 (SIR). *The SIR is the fraction of candidate paths pruned by the oracle:*

$$\text{SIR} = \frac{|\{p \in \mathcal{P} : O(p) = 0\}|}{|\mathcal{P}|} = \text{tightness}(O)$$

Our oracle achieves $\text{SIR} = 14.5\%$ on WebQSP.

9 Beam Search

9.1 Beam unit

Definition 9.1 (Beam). A **beam** in $v2$ is a tuple $b = (\sigma, H, s, t)$ where:

- σ : the generated sequence so far,
- $H \subseteq \mathcal{E}$: the head pool (set of entities available for the next hop),
- $s \in \mathbb{R}$: cumulative score (sum of log-probabilities),
- $t \in \mathbb{N}$: current hop number.

9.2 Beam scoring

Definition 9.2 (Total score). The total score of beam b is:

$$\text{score}(b) = \sum_{i=1}^t s_i$$

where s_i is the score assigned by the model at hop i .

Beams are ranked by total score; the top- B beams survive to the next hop.

9.3 Head pool update

Definition 9.3 (Head pool update). After generating a hop from entity e to entity e' , the head pool updates as:

$$H_{\text{new}} = H_{\text{old}} \cup \{e'\}$$

This is Python set union: `new_head_pool = beam.head_pool | {new_entity}`. The new entity is added to the pool, making it available as a starting point for the next hop.

10 The Faithfulness Theorem

Theorem 10.1 (Faithfulness of ontology-constrained decoding). Let $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ be a knowledge graph, O_{ont} be the ontology oracle, and **Trie** be the trie constructed from paths admitted by O_{ont} . For any token sequence $\sigma = (t_1, \dots, t_n)$ generated by the constrained decoder with prefix allowed tokens from **Trie**, every prefix $\sigma_{1:k}$ for $k \leq n$ corresponds to a valid path in $\mathcal{G}$.

Proof sketch. By induction on the trie depth k .

Base case ($k = 0$): The empty prefix corresponds to the empty path. Vacuously valid.

Inductive step: Assume $\sigma_{1:k}$ corresponds to a valid path $(e_0, r_1, e_1, \dots, r_j, e_j)$ in $\mathcal{G}$. At step $k+1$, the trie returns only tokens t such that $\sigma_{1:k} \oplus t$ is a prefix of some path in the trie. Each such path corresponds to a valid KG path (by construction of the trie from $\mathcal{P}_{\text{filtered}}$). Therefore $\sigma_{1:k+1}$ corresponds to a valid path.

Key invariant: The trie is built from paths in $\mathcal{P}_{\text{filtered}} \subseteq \mathcal{P}_{\text{GCR}} \subseteq \{\text{valid KG paths}\}$. Every prefix of a valid KG path is itself a valid KG path. The trie preserves this prefix-closure property. $\square$

Why this matters. The theorem guarantees that the ontology oracle does not introduce hallucination. Every generated token sequence is a valid KG path. The oracle only *removes* paths from the constraint set; it never adds new ones. Since removal preserves prefix-closure, faithfulness is maintained.

11 SIR Type Component

Definition 11.1 ($\text{SIR}_{\text{type}}^*$). *For a set of candidate paths $\mathcal{P}$, oracle O , and answer types $\mathcal{T}(q)$:*

$$\text{SIR}_{\text{type}}^* = \frac{|\{p \in \mathcal{P} : O_{\text{type}}(p) = 0\}|}{|\mathcal{P}|}$$

This measures the fraction of paths pruned by the type gate alone. It isolates the type gate’s contribution to the overall SIR.

12 Adaptive Enumeration Budget

Definition 12.1 (Question complexity). *For question q , define the feature vector:*

$$f(q) = (n_{\text{start}}, n_{\text{types}}, n_{\text{neighbors}})$$

where n_{start} is the number of start entities, n_{types} is the number of inferred answer types, and $n_{\text{neighbors}}$ is the average out-degree.

Definition 12.2 (Budget allocation).

$$K(q) = \begin{cases} 30 & n_{\text{start}} \leq 1 \wedge n_{\text{types}} \leq 2 \wedge n_{\text{neighbors}} \leq 20 \\ 100 & n_{\text{start}} \leq 3 \wedge n_{\text{types}} \leq 5 \wedge n_{\text{neighbors}} \leq 100 \\ 500 & \text{otherwise} \end{cases}$$

Easy questions get a small budget (fast); hard questions get a large budget (accurate).

13 Embedding Initialization

When adding new tokens to a pretrained model, we initialize their embeddings as the average of existing embeddings:

Definition 13.1 (Average initialization). *For n_{new} new tokens, let $\{e_1, \dots, e_{|V|}\}$ be the existing embeddings. The new embeddings are:*

$$e_{\text{new}} = \frac{1}{|V|} \sum_{i=1}^{|V|} e_i$$

This places new tokens at the centroid of the embedding space, giving them a neutral starting point.

14 Auto-Mining Relation Schema

Definition 14.1 (Mined schema). *For relation r , the mined domain and range are:*

$$\text{domain}_{\text{mined}}(r) = \bigcup_{(h,r,t) \in \mathcal{T}} \text{types}(h)$$

$$\text{range}_{\text{mined}}(r) = \bigcup_{(h,r,t) \in \mathcal{T}} \text{types}(t)$$

For each triple with relation r , we look up the head entity’s types (contributing to domain) and the tail entity’s types (contributing to range). The union aggregates across all triples.

Why auto-mine? The hand-curated schema covers 40 relations. Auto-mining covers ALL relations in the subgraph, increasing coverage. The mined schema is used as a fallback when the hand-curated schema does not contain the relation.

15 Summary of Notation

Symbol	Meaning
$\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$	Knowledge graph
$\mathcal{E}$	Entity set
$\mathcal{R}$	Relation set
$\mathcal{T}$	Triple set
E_q	Topic entity for question q
L	Hop depth (number of hops)
p	Reasoning path
p_q^*	Gold-truth path for question q
$\mathcal{P}$	Set of candidate paths
$\mathcal{C}$	Universe of type constants
$\text{types}(e)$	Type set of entity e
$\text{range}(r)$	Range type set of relation r
$\mathcal{T}(q)$	Answer type set for question q
O	Constraint oracle
$\text{tightness}(O)$	Fraction of paths excluded by O
O_{GCR}	Structural oracle (admits everything)
O_{ont}	Ontology oracle (our contribution)
ϕ	Filter function
$E(\cdot)$	Sentence embedding function
$\cos(u, v)$	Cosine similarity
τ	Admission threshold
B	Beam width
d_{avg}	Average out-degree
W_t	Admissible token set at step t
$\sigma_{1:t}$	Generated token sequence up to step t
L_{input}	Position where constrained decoding starts

16 How to Explain Each Formula

This section provides one-sentence explanations for each key formula, suitable for a viva or defense.

Knowledge graph. “A knowledge graph is a set of entities, relations, and triples connecting them.”

Path. “A reasoning path is a sequence of entities and relations starting from the question topic.”

Type gate. “The type gate prunes terminal entities whose types don’t match what the question is asking for.”

Range gate. “The range gate prunes tail entities whose types are incompatible with the relation’s declared range.”

Oracle tightness. “Tightness measures what fraction of paths the oracle removes—higher means more aggressive pruning.”

Conservative default. “When metadata is missing, we pass rather than prune, to avoid false negatives.”

Trie. “A trie encodes all valid paths as a prefix tree, enabling $O(1)$ next-token lookup during decoding.”

Faithfulness theorem. “The ontology oracle only removes paths, never adds new ones, so every generated sequence is still a valid KG path.”

Cosine similarity. “Cosine similarity measures how aligned two vectors are, regardless of their magnitude.”

Decomposed score. “The decomposed score multiplies three independent signals: relation relevance, type compatibility, and trajectory relevance.”

Beam search. “Beam search keeps the B most promising partial paths at each step, balancing exploration and exploitation.”

Head pool. “The head pool tracks all entities mentioned so far, enabling the model to branch from any of them in the next hop.”

SIR. “SIR measures the oracle’s selectivity—what fraction of the path space it removes.”

V1 vs V2 complexity. “V1 is exponential in hop depth; V2 is linear, because it only enumerates one hop at a time.”